鲲鹏 920 专业版系统开发手册-V1.1

文档版本 01

发布日期 2026-06-01



华为技术有限公司

版权所有 © 华为技术有限公司 2026。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明


和其他华为商标均为华为技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司

地址： 深圳市龙岗区坂田华为总部办公楼 邮编：518129

网址： https://www.huawei.com

客户服务邮箱： support@huawei.com

客户服务电话： 4008302118

安全声明

漏洞处理流程

华为公司对产品漏洞管理的规定以“漏洞处理流程”为准，该流程的详细内容请参见如下网址：
https://www.huawei.com/cn/psirt/vul-response-process
如企业客户须获取漏洞信息，请参见如下网址：
https://securitybulletin.huawei.com/enterprise/cn/security-advisory

 版权所有 © 华为技术有限公司

目 录

1 前言.....1
2 鲲鹏 920 专业版介绍.....2
2.1 产品描述.....2
2.1.1 简介.....2
2.1.2 产品特点.....3
2.2 目标用户.....4
3 鲲鹏 920 专业版架构.....5
3.1 物理结构.....5
3.2 逻辑结构.....6
3.3 浮点计算加速.....6
3.3.1 向量计算介绍.....6
3.3.2 矩阵计算介绍.....6
4 鲲鹏 920 专业版片上内存.....7
4.1 片上内存介绍.....7
4.2 Flat 模式下片上内存编程.....7
5 鲲鹏 920 专业版配置.....10
5.1 大页内存配置.....10
5.2 BIOS 配置.....13
6 openEuler 系统配置.....16
6.1 openEuler 核隔离配置.....16
6.2 代码段多副本配置.....17
6.2.1 普通代码段多副本配置步骤.....18
6.2.2 Python 代码段多副本配置步骤.....20
7 鲲鹏 920 专业版矩阵算力编程.....22
7.1 矩阵编程指导.....22
7.2 毕昇自动使能矩阵计算加速.....23
7.3 对接 HPCKit 加速库使能矩阵计算加速.....23
7.3.1 对接鲲鹏数学库.....23
7.3.2 对接 KuDNN 算子库.....24
7.4 对接 KUPL MMA 使能矩阵计算加速.....25
7.5 ARM Intrinsic 使能矩阵计算加速.....26

 版权所有 © 华为技术有限公司

8 鲲鹏 920 专业版 SDMA 编程 27
8.1 SDMA 介绍及使能 27
8.2 基于 KUPL 实现 27
9 鲲鹏 920 专业版负载测试 29
9.1 Benchmark 测试 29
9.1.1 DGEMM 测试 29
9.1.1.1 工具介绍 29
9.1.1.2 工具安装 29
9.1.1.3 测试运行 30
9.1.2 STREAM 测试 31
9.1.2.1 工具介绍 31
9.1.2.2 工具安装 31
9.1.2.3 片上内存带宽测试 32
9.2 应用分析 34
9.2.1 片上内存使用分析与性能优化 34
9.2.2 矩阵算力使用分析与性能优化 36

 版权所有 © 华为技术有限公司

1 前言

概述

本文档详细描述了鲲鹏920专业版的参数，提供了常见的系统配置方法及使用指南。

读者对象

本文档主要适用于初次使用鲲鹏920专业版的人员。人员必须具备以下经验和技能：

- 熟悉Linux系统，有Linux程序编程经验。

 版权所有 © 华为技术有限公司

2 鲲鹏 920 专业版介绍

2.1 产品描述
2.2 目标用户

2.1 产品描述

2.1.1 简介

TaiShan 200 服务器是天池架构下基于华为鲲鹏920处理器的数据中心服务器，其中，鲲鹏920专业版对应的服务器是TaiShan 200服务器的一种特殊型号。该服务器面向互联网、分布式存储、云计算、大数据、企业业务等领域，具有高性能计算、大容量存储、低能耗、易管理、易部署等优点。其外观图如**图2-1**所示。

图 2-1 外观图



 版权所有 © 华为技术有限公司

2.1.2 产品特点

性能和扩展特点

- 支持华为自研的、面向服务器领域的≥64bits多核鲲鹏920处理器，内部集成了DDR、PCIe 5.0、200GE、100GE、25GE、10GE、GE等接口，提供完整的SoC功能。
- 单台服务器支持2个处理器，单个处理器拥有多个物理核心，能够最大限度地提高多线程应用的并发执行能力。
- 支持多种灵活的硬盘配置方案，提供了弹性的、可扩展的存储空间，满足不同存储空间的需求和升级要求。
- 支持灵活插卡，可提供多种以太网卡接口能力。
- 可支持≥6个PCIe标准扩展槽位。

众核特点

- 单颗处理器支持304个物理核心（CPU core），支持多NUMA架构。
- 每个CPU core内集成私有的32KB L1 I-Cache，32KB L1 D-Cache以及768KB L2 Cache。
- 38个CPU cores形成一个CPU Cluster，即一个NUMA；4个 CPU Cluster组成一个Super CPU Cluster（SCCL），即152个CPU Cores；
- 通过多DIE合封技术，一个SCCL封装为一个DIE，单颗处理器最多可合封装2个DIE，即2个SCCL（8个CPU Cluster或8个NUMA）

可用性和可服务性特点

- 支持SAS/SATA/NVMe硬盘，其中SAS/SATA硬盘可根据不同的RAID控制卡设置不同的RAID级别，可提供RAID Cache，详细内容请参见**《RAID控制卡 用户指南(Arm)》**。支持超级电容掉电数据保护，支持非系统硬盘热插拔。
- 通过面板提供UID/HLY LED指示灯，iBMC Web管理界面提供关键部件指示状态能够指引技术人员快速找到已经发生故障（或者正在发生故障）的组件，从而简化维护工作、加快解决问题的速度，并且提高系统可用性。
- BMC集成管理模块（iBMC）能够持续监测系统参数、触发告警，并且采取恢复措施，以便最大限度地避免停机。

可管理性及安全性特点

- 集成在服务器上的iBMC管理模块可用来监测系统运行状态，并提供远程管理功能。
- 集成了业界标准的统一可扩展固件接口（UEFI），因此能够提高设置、配置和更新效率，并且简化错误处理流程。
- 支持带锁的服务器机箱安全面板，保护服务器本地数据的安全。

能源效率

- 提供钛金电源模块，50%负载下电源模块能量转换效率最高可达96%。
- 高效率的单板VRD电源，降低DC转DC的损耗。
- 支持主备供电。

 版权所有 © 华为技术有限公司

- 支持PID（Proportional-Integral-Derivative）智能调速，节能降耗。
- 全方面优化的系统散热设计，高效节能系统散热风扇，降低系统散热能耗。
- 硬盘错峰上电技术，降低服务器启动功耗。
- 支持SSD硬盘，SSD硬盘的功耗比传统机械硬盘低80%。

2.2 目标用户

本文档主要适用于初次使用鲲鹏920专业版的人员。人员必须具备以下经验和技能：

- 熟悉Linux系统，有Linux程序编程经验。

 版权所有 © 华为技术有限公司

3 鲲鹏 920 专业版架构

本章以其中一款服务器型号为例，介绍鲲鹏920专业版对应的服务器架构。

3.1 物理结构
3.2 逻辑结构
3.3 浮点计算加速

3.1 物理结构

鲲鹏920专业版服务器提供16个内存槽，本章节以8盘配置为例，描述服务器的物理结构。

表 3-1 部件说明

序号	部件名称	序号	部件名称
1	IO模组1	2	IO模组2
3	IO模组3挡板	4	电源模块
5	机箱	6	CPU散热器
7	风扇板	8	前置硬盘背板
9	风扇支架	10	风扇模块
11	前置硬盘	12	理线架
13	基础板	14	CPU托架
15	DIMM	16	超级电容
17	扩展板	18	OCP导风罩
19	FlexIO卡1（归属CPU1）	20	BMC插卡
21	FlexIO卡2（归属CPU2）	22	扩展板理线架

 版权所有 © 华为技术有限公司

说明

- CPU集成在基础板上，不能单独更换。
- 备件的详细信息请联系华为技术支持参见**备件查询**。

3.2 逻辑结构

本产品支持Hi1711 BMC插卡，可外出VGA、管理网口、串口、USB等管理接口。

- 支持两路华为自研鲲鹏920处理器，每个处理器可支持8个DDR内存。
- 两路处理器之间通过2组HCCS总线互连，传输速率≥30Gbps。
- 处理器的PCIe资源，通过PCB或者线缆与PCIe Riser卡连接。
- Riser卡支持不同规格的PCIe槽位。CPU1和CPU2各支持1张Flex IO网卡，可支持2*25/10GE。
- 独立BMC管理芯片，支持外出VGA、管理网口、串口等接口。

3.3 浮点计算加速

鲲鹏920专业版的处理器支持512bit的向量位宽，意味着单条指令可以同时处理更多数据，同时既支持向量计算也支持矩阵计算，这使得计算吞吐量倍增，理论算力也更高。

3.3.1 向量计算介绍

仅在向量计算加速使能的情况下，支持向量的内积操作。以双精度浮点数为例，一个向量长度可容纳8个双精度浮点数，执行一次向量内积支持乘累加两个操作，同时单cycle支持两次向量内积的乘累加操作，因此每周期可执行32次浮点运算。对于不同数据精度，向量算力比值分别是FP64:FP32:FP16:int8=1:2:4:8。

3.3.2 矩阵计算介绍

仅在矩阵计算加速使能的情况下，支持向量的外积操作。以双精度浮点数为例，一个向量长度可容纳8个双精度浮点数，执行一次向量外积支持乘累加两个操作，同时单cycle支持一次向量外积的乘累加操作，因此每周期可执行128次浮点运算。对于不同数据精度，矩阵算力比值分别是FP64:FP32:FP16:int8=1:2:4:16。

 版权所有 © 华为技术有限公司

4 鲲鹏 920 专业版片上内存

4.1 片上内存介绍
4.2 Flat模式下片上内存编程

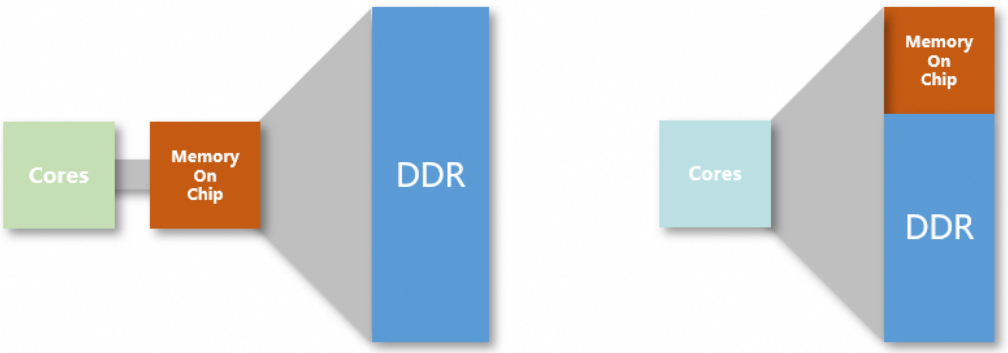
4.1 片上内存介绍

鲲鹏920专业版片上内存有两种模式可供选择。

图4-1中左侧是Cache模式，右侧是Flat模式。

- Cache模式下用户不感知片上内存，无需针对片上内存编写代码，计算数据从DDR到片上内存再到Cores。Cache模式在快速迁移上更具优势，无需用户修改代码。
- Flat模式下DDR和片上内存作为独立的NUMA Node对用户可见，由用户指定计算数据从片上内存到Cores还是从DDR到Cores。Flat模式在性能极致调优上更具优势，需要用户修改代码。

图 4-1 Cache 模式和 Flat 模式示意图

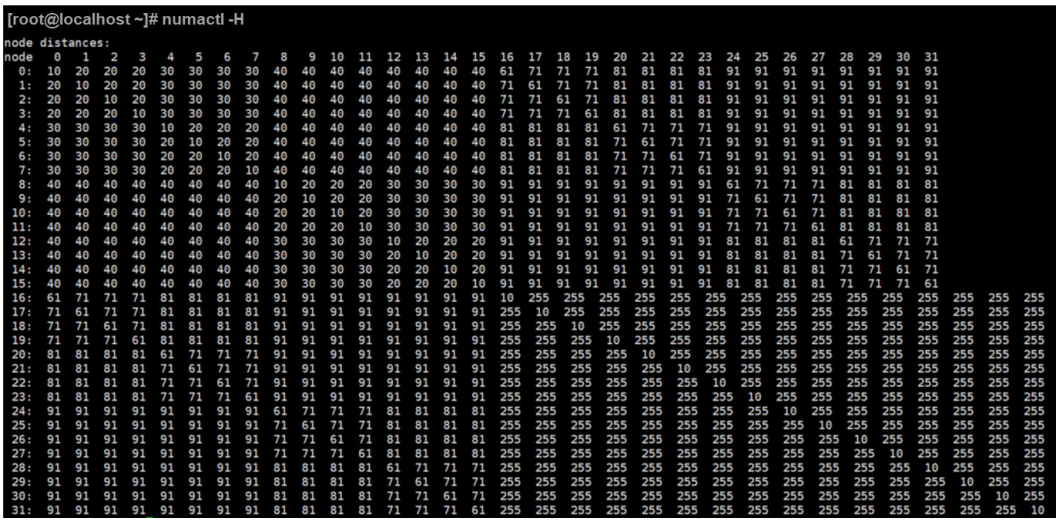


4.2 Flat 模式下片上内存编程

Flat模式下DDR和片上内存作为独立的NUMA Node对用户可见，执行“**numactl -H**”命令，查看输出结果，其中0-15行是DDR（普通内存），16-31行是片上内存。

 版权所有 © 华为技术有限公司

图 4-2 节点的 DDR 和片上内存分布



方法一：使用 malloc 接口申请内存

该方法是程序使用malloc接口申请内存，运行程序时使用numactl命令绑定至NUMA Node的片上内存。该方法可以在程序无修改情况下，透明使用片上内存，是应用内存容量需求小于片上内存容量时的首选策略。

- 强制从指定的0 NUMA Node的片上内存分配内存，如果程序超出指定NUMA Node的容量，程序报错Out of Memory并中止。
  - **numactl -m** *memory_on_chip_node1 ./a.out*
- 程序首先从指定的NUMA Node的片上内存开始分配内存，片上内存分配完后，会使用该Node的DDR内存继续分配。
  - **numactl --preferred** *memory_on_chip_node1 ./a.out*

说明

memory_on_chip_node1 表示指定的节点ID

*a.out 表示*要运行的程序的可执行文件

以下以启动560个进程，一个进程运行在一个核上举例：

mpirun -np *560* --rankfile rfile run_on_package_memory_node.sh

```
#!/bin/sh

memid=`expr ${OMPI_COMM_WORLD_LOCAL_RANK} / 35`  # 每35个进程跑在一个numa上，就可以确定当前进程在第几个numa上
memid=`expr $memid + 16`                         # 放在图1 节点的DDR和片上内存分布的17行，也就是指定为片上内存模式
numactl -m ${memid} ./a.out                      # a.out表示运行的二进制
```

方法二：使用 KUPL 接口申请片上内存

通过替换malloc函数来实现申请片上内存，举例如下

```
#include <kupl.h>

float *x = (float *)kupl_hbw_malloc(N*sizeof(float));

kupl_hbw_free(x);
```

 版权所有 © 华为技术有限公司

方法三：使用 memkind 接口申请片上内存

通过替换malloc函数来实现申请片上内存，举例如下

```
#include <hbwmalloc.h>

float *x = (float *)hbw_malloc(N*sizeof(float));

hbw_free(x);
```

 版权所有 © 华为技术有限公司

5 鲲鹏 920 专业版配置

5.1 大页内存配置

5.2 BIOS配置

5.1 大页内存配置

大页内存介绍

在页式虚拟存储器中，会把虚拟存储空间和物理主存空间都分割为一个个固定大小的页，为线程分配内存也是以页为单位。比如：页的大小为 4K，那么 4GB 存储空间就需要 4GB/4KB=1M 条记录，即有 100 多万个 4KB 的页。如果页太小了，那么就会产生大量的页表条目，降低了查询速度的同时还浪费了存放页面的主存空间；但如果页太大了，又会容易造成浪费，原因就跟段式存储管理方式一样。所以 Linux 操作系统默认的页大小就是 4KB，可以通过指令查看：

```
$ getconf PAGE_SIZE
4096
```

但在某些对性能要求非常苛刻的场景中，页面会被设置得非常的大，比如：2MB、1GB、甚至几十 GB，这些页被称之为 “大页”（ Huge Page ）。

大页能够提升性能的主要原因有以下几点：

- 减少页表条目，加快检索速度。
- 提升 TLB 快表的命中率，TLB 一般拥有 16 ~ 128 个条目空间，也就是说当大页为 1GB 的时候，TLB 能够对应 16GB ~ 128GB 之间的存储空间。

大页内存配置步骤

大页内存配置需要连续的内存空间，可以在Linux中动态配置大页内存，实现实时开关，整体步骤如下。

步骤1 查看Linux默认的大页内存配置大小。openEuler有2MB、1GB两种大页内存配置，默认为2MB大页内存，若需要修改大页内存为1GB配置需重编OS内核，当前配置均以2MB大页内存进行示例。

```
$ grep -i hugepagesize /proc/meminfo
Hugepagesize:       2048 kB
```

步骤2 查看 Linux 操作系统是否启动了大页内存，如果 HugePages_Total 为 0，意味着 Linux 没有设置或没有启用 Huge pages。

当前鲲鹏920专业版上每个节点共2个CPU，每个CPU 8个NUMA，共16个NUMA。每个NUMA有32GB DDR内存和4GB 片上内存，可通过numastat -m命令查看节点信息，可以看到Node 0-15即为NUMA的DDR内存情况，Node 16-31为NUMA的片上内存情况。各Node下“HugePages_Total”为大页内存大小，可以看到均为空

```
                 Node 0          Node 1          Node 2          Node 3
---------------- --------------- --------------- --------------- ---------------
MemTotal          31376.38        32250.62        32250.62        32250.62
MemFree           26903.30        27613.54        31664.97        31624.89
MemUsed            4473.07         4637.07          585.65          625.73
Active              137.05          120.04            6.26            6.46
Inactive           2128.45         2102.39           15.93           71.89
Active(anon)        137.05          119.78            6.25            6.39
Inactive(anon)     2127.74         2101.00           14.87           70.76
Active(file)          0.00            0.27            0.01            0.06
Inactive(file)        0.71            1.39            1.07            1.14
Unevictable           0.00            0.00            0.00            0.00
Mlocked               0.00            0.00            0.00            0.00
Dirty                 0.00            0.00            0.00            0.00
Writeback             0.00            0.29            0.15            0.38
FilePages          2203.80         2185.45            9.27            9.66
Mapped              357.91          293.75          239.99          211.84
AnonPages            61.55           37.23           11.53           67.19
Shmem              2202.66         2183.79            8.20            8.46
KernelStack          17.89            5.08            4.95            5.38
PageTables            0.25            0.80            0.00            0.46
NFS_Unstable          0.00            0.00            0.00            0.00
Bounce                0.00            0.00            0.00            0.00
WritebackTmp          0.00            0.00            0.00            0.00
Slab                563.59         1289.68           39.69           51.61
SReclaimable          3.46            7.61            6.14            9.29
SUnreclaim          560.13         1282.07           33.55           42.32
AnonHugePages         0.00          156.00           20.00          248.00
ShmemHugePages        0.00            0.00            0.00            0.00
ShmemPmdMapped        0.00            0.00            0.00            0.00
HugePages_Total       0.00            0.00            0.00            0.00
HugePages_Free        0.00            0.00            0.00            0.00
HugePages_Surp        0.00            0.00            0.00            0.00
KReclaimable          3.46            7.61            6.14            9.29
...
...
...
                 Node 16         Node 17         Node 18         Node 19
---------------- --------------- --------------- --------------- ---------------
MemTotal          4096.00         4096.00         4096.00         4096.00
MemFree           4096.00         4096.00         4096.00         4096.00
MemUsed              0.00            0.00            0.00            0.00
Active               0.00            0.00            0.00            0.00
Inactive             0.00            0.00            0.00            0.00
Active(anon)         0.00            0.00            0.00            0.00
Inactive(anon)       0.00            0.00            0.00            0.00
Active(file)         0.00            0.00            0.00            0.00
Inactive(file)       0.00            0.00            0.00            0.00
Unevictable          0.00            0.00            0.00            0.00
Mlocked              0.00            0.00            0.00            0.00
Dirty                0.00            0.00            0.00            0.00
Writeback            0.00            0.00            0.00            0.00
FilePages            0.00            0.00            0.00            0.00
Mapped               0.00            0.00            0.00            0.00
AnonPages            0.00            0.00            0.00            0.00
Shmem                0.00            0.00            0.00            0.00
KernelStack          0.00            0.00            0.00            0.00
PageTables           0.00            0.00            0.00            0.00
NFS_Unstable         0.00            0.00            0.00            0.00
Bounce               0.00            0.00            0.00            0.00
```

 版权所有 © 华为技术有限公司

```
WritebackTmp              0.00          0.00          0.00          0.00
Slab                    0.00          0.00          0.00          0.00
SReclaimable              0.00          0.00          0.00          0.00
SUnreclaim                0.00          0.00          0.00          0.00
AnonHugePages             0.00          0.00          0.00          0.00
ShmemHugePages            0.00          0.00          0.00          0.00
ShmemPmdMapped            0.00          0.00          0.00          0.00
HugePages_Total           0.00          0.00          0.00          0.00
HugePages_Free            0.00          0.00          0.00          0.00
HugePages_Surp            0.00          0.00          0.00          0.00
KReclaimable              0.00          0.00          0.00          0.00
...
...
...
                          Total
                ---------------
MemTotal              578560.55
MemFree               555020.91
MemUsed                23539.65
Active                   363.48
Inactive                6470.92
Active(anon)             362.12
Inactive(anon)          6453.55
Active(file)               1.35
Inactive(file)            17.36
Unevictable              128.20
Mlocked                  128.20
Dirty                      0.21
Writeback                  4.21
FilePages               4754.86
Mapped                  3560.32
AnonPages               2200.26
Shmem                   4735.73
KernelStack               98.20
PageTables                22.99
NFS_Unstable               0.00
Bounce                     0.00
WritebackTmp               0.00
Slab                    3117.82
SReclaimable             188.76
SUnreclaim              2929.06
AnonHugePages           3048.00
ShmemHugePages             0.00
ShmemPmdMapped             0.00
HugePages_Total            0.00
HugePages_Free             0.00
HugePages_Surp             0.00
KReclaimable             188.76
```

步骤3 开启大页内存

```
#!/bin/bash
for i in {0..31}
do
  # 开启2000个大页2MB大页
  echo 2000 > /sys/devices/system/node/node$i/hugepages/hugepages-2048kB/nr_hugepages
done
```

步骤4 查看大页内存

```
$ numastat -m | grep  HugePages_Total
HugePages_Total           0.00       4000.00       4000.00       4000.00       4000.00       4000.00
4000.00       4000.00       4000.00       4000.00       4000.00       4000.00       4000.00
4000.00       4000.00       4000.00       4000.00       4000.00       4000.00       4000.00
4000.00       4000.00       4000.00       4000.00       4000.00       4000.00       4000.00
4000.00       4000.00       4000.00       4000.00       4000.00     124000.00
```

步骤5 关闭大页内存

```
#!/bin/bash
for i in {0..31}
do
```

 版权所有 © 华为技术有限公司

```
 echo 0 > /sys/devices/system/node/node$i/hugepages/hugepages-2048kB/nr_hugepages
done

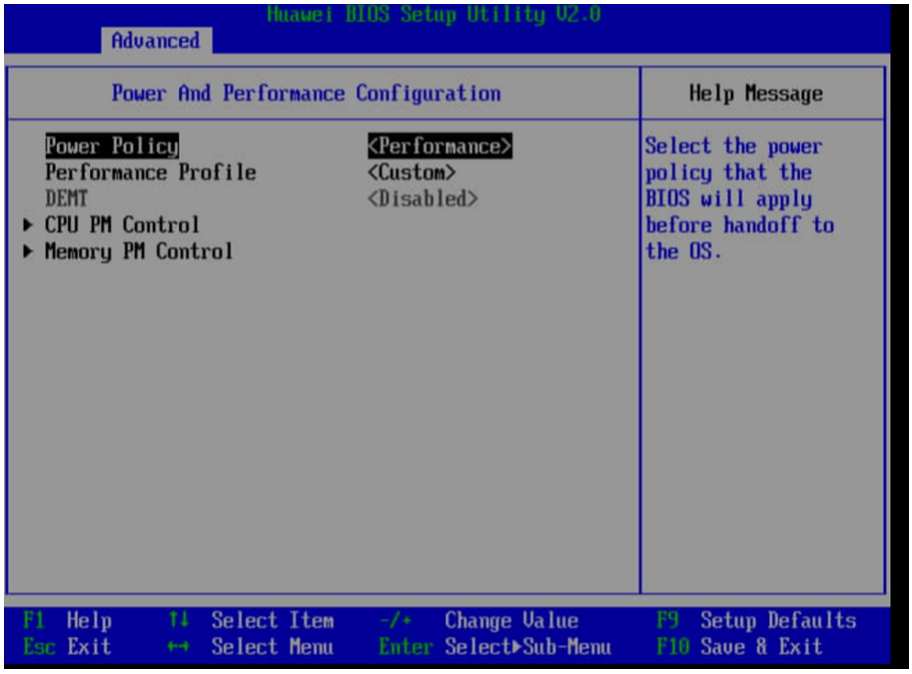
# 关闭后查看大页内存情况
$ numastat -m | grep  HugePages_Total
HugePages_Total          0.00          0.00          0.00          0.00          0.00          0.00
0.00          0.00          0.00          0.00          0.00          0.00          0.00          0.00
0.00          0.00          0.00          0.00          0.00          0.00          0.00          0.00
0.00          0.00          0.00          0.00          0.00          0.00          0.00          0.00
0.00          0.00          0.00
```

----结束

5.2 BIOS 配置

- **能效模式配置**

步骤1 进入BIOS配置界面，依次点击Advanced->Power And Performance Configuration



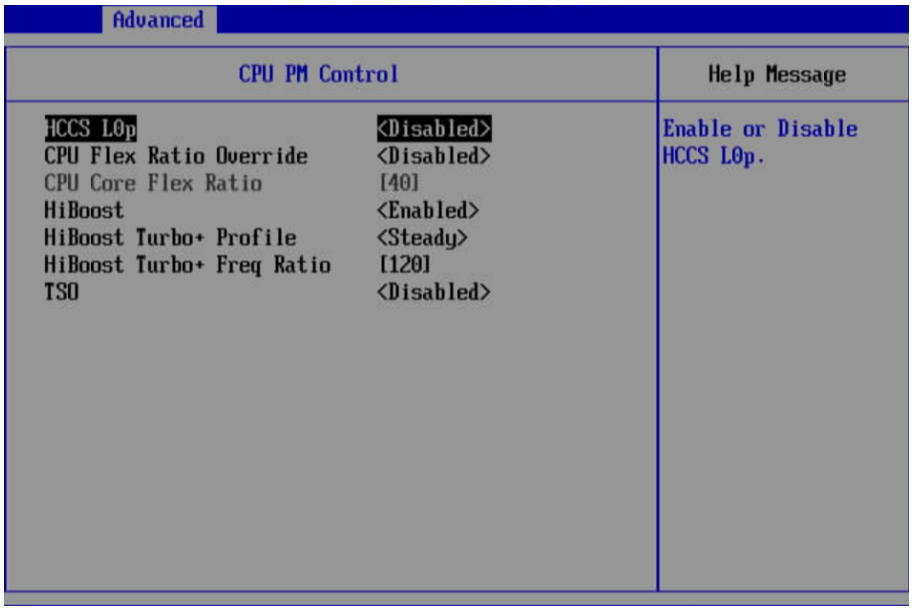
说明

1. Performance Profile（能效模式）：设置为Custom，设置能效模式为自定义模式
2. Power Policy（通电策略）：设置为Performance，设置通电策略为性能模式
3. DEMT（智能限耗）：随Power Policy的设置为Performance而自动变为Disable，即关闭智能限耗启动性能模式

----结束

- **超频配置**

步骤1 进入BIOS配置界面，依次点击Advanced->Power And Performance Configuration->CPU PM Control



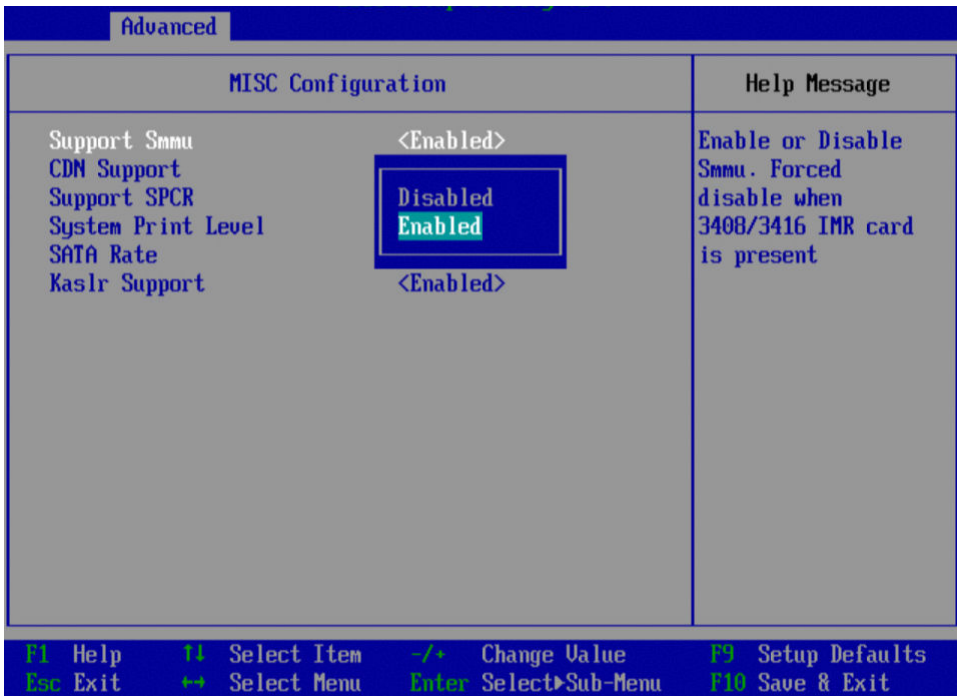
说明

1. HiBoost（超频配置）：设置为Enabled，开启超频模式
2. HiBoost Turbo+ Profile（超频模式）：设置为Steady模式，即会始终固定在最高频率，除非功耗和温度超限
3. Freq Ratio（超频频率比例）：设置为最大值120，即采用最大超频上限

----结束

- **SMMU配置**

步骤1 进入BIOS配置界面，依次点击Advanced->MISC Configuration



 版权所有 © 华为技术有限公司

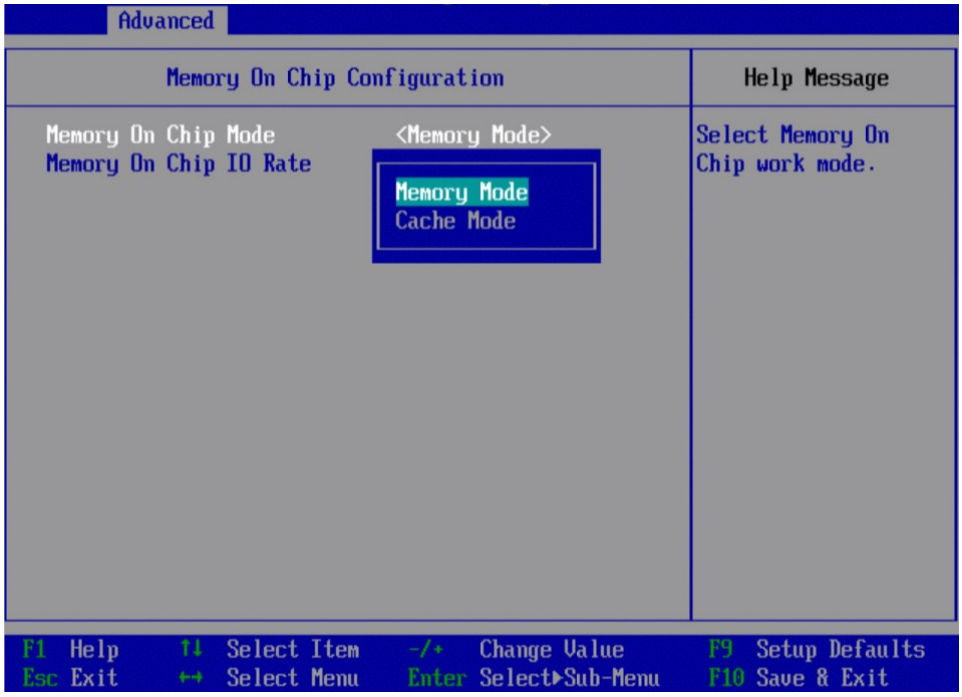
说明

1. Support Smmu（SMMU配置）：设置为Enabled，开启Smmu以启动SDMA设备

----结束

- **片上内存模式设置**

步骤1 进入BIOS配置界面，依次点击Advanced->Memory Configuration->Memory On Chip Configuration



说明

1. Memory On Chip Mode（内存模式配置）：设置片上内存模式为Flat模式或Cache模式

----结束

 版权所有 © 华为技术有限公司

6 openEuler 系统配置

6.1 openEuler核隔离配置

6.2 代码段多副本配置

6.1 openEuler 核隔离配置

openEuler 核隔离介绍

在HPC场景下，由于每个线程会频繁进行同步，OS背景噪声对性能的影响会随着节点数增多逐步放大，OS背景噪声包括背景守护进程、外设中断、内核背景线程等。

增强核隔离特性将系统CPU分为housekeeping CPU和non-housekeeping CPU，将OS背景噪声集中在housekeeping CPU上，non-housekeeping CPU只运行业务计算任务，通过减少业务运行时背景噪声的干扰，提升业务性能。non-housekeeping CPU可以通过启动参数nohz_full和isolcpus指定，增加enhanced_isolcpus参数后可以进一步消除磁盘IO的噪声干扰。

核隔离配置步骤

步骤1 修改grub启动项。

vim /boot/efi/EFI/openEuler/grub.cfg

找到关键字“/vmlinuz”所在行，在行末尾添加以下内容：

```
irqaffinity=37,75,113,151,189,227,265,303,341,379,417,455,493,531,569,607
nohz_full=0-36,38-74,76-112,114-150,152-188,190-226,228-264,266-302,304-340,342-378,380-416,418-454,456-492,494-530,532-568,570-606
isolcpus=nohz,domain,managed_irq,0-36,38-74,76-112,114-150,152-188,190-226,228-264,266-302,304-340,342-378,380-416,418-454,456-492,494-530,532-568,570-606
rcu_nocbs=0-36,38-74,76-112,114-150,152-188,190-226,228-264,266-302,304-340,342-378,380-416,418-454,456-492,494-530,532-568,570-606
disable_sdei_nmi_watchdog enhanced_isolcpus
```

当前鲲鹏920专业版上每个节点共2个CPU，每个CPU 8个NUMA，每个NUMA 38核，整机共608核，核隔离参数推荐将每个NUMA的最后1个核配置为housekeeping CPU，housekeeping CPU的优先级为“irqaffinity”，使内核进程/中断优先调度到housekeeping CPU上，减少系统对non-housekeeping CPU的影响。

 版权所有 © 华为技术有限公司

同时，因配置核隔离后，若程序使用MPI顺序绑核会导致部分计算进程在housekeeping CPU上运行，影响程序整体性能。在该配置下，MPI可以指定rankfile，显式指定进程绑定的核。

步骤2 重启节点，使核隔离参数生效。

步骤3 重启后执行 **cat /proc/cmdline**，确认参数是否添加成功。

步骤4 执行初始化脚本，将系统中断/服务绑定到housekeeping CPU，并进行性能配置。

```
#!/bin/bash

# 禁用自动优化Linux系统硬件中断CPU分配的服务，强制绑定到housekeeping CPU
systemctl stop irqbalance && systemctl mask irqbalance

#设置实时任务可以使用全部CPU时间
echo -1 > /proc/sys/kernel/sched_rt_runtime_us

#设置khugepaged 后台线程在合并页面时的碎片整理行为为不整理碎片
echo 0 >  /sys/kernel/mm/transparent_hugepage/khugepaged/defrag

#设置进程分配内存时的即时行为为从不整理碎片
echo never > /sys/kernel/mm/transparent_hugepage/defrag

#设置透明大页内存分配策略为系统全局开启
echo always > /sys/kernel/mm/transparent_hugepage/enabled

#禁用自动NUMA平衡
echo 0 > /proc/sys/kernel/numa_balancing

ps -aux | grep rcu_sched | grep -v 'grep' | awk  '{print $2}' > ~/.tmp
 cat ~/.tmp | while read line; do taskset -pc 37,75,113,151,189,227,265,303,341,379,417,455,493,531,569,607
$line; done

 ps -aux | grep kswapd | grep -v 'grep' | awk  '{print $2}' > ~/.tmp
 cat ~/.tmp | while read line; do taskset -pc 37,75,113,151,189,227,265,303,341,379,417,455,493,531,569,607
$line; done

 ps -aux | grep kcompactd  | grep -v 'grep' | awk  '{print $2}' > ~/.tmp
 cat ~/.tmp | while read line; do taskset -pc 37,75,113,151,189,227,265,303,341,379,417,455,493,531,569,607
$line; done

 ps -aux | grep rcuog | grep -v 'grep' | awk  '{print $2}' > ~/.tmp
 cat ~/.tmp | while read line; do taskset -pc 37,75,113,151,189,227,265,303,341,379,417,455,493,531,569,607
$line; done

 ps -aux | grep rcuos | grep -v 'grep' | awk  '{print $2}' > ~/.tmp
 cat ~/.tmp | while read line; do taskset -pc 37,75,113,151,189,227,265,303,341,379,417,455,493,531,569,607
$line; done

# 配置所有核的模式为performance
for core_id in 0-607
do
   echo performance > /sys/devices/system/cpu/cpufreq/policy${core_id}/scaling_governor
done
```

----结束

6.2 代码段多副本配置

代码段多副本介绍

非统一内存访问NUMA (Non-Uniform Memory Access) 是一种计算机架构设计，920专业版服务器属于NUMA架构，不同的处理器核心被称为节点，拥有自己的本地内

存，当一个NUMA节点需要访问其他NUMA节点的代码段时，会因此产生额外的时延和性能开销。

多副本功能的核心作用是当出现上述跨节点访问时，提前在当前节点上创建好待访问的代码段，从跨节点获取转换成本地获取，从而减少跨节点带来的时延和性能开销。

普通代码段多副本适用于C/C++/Fortran等语言编译的二进制文件，Python代码段多副本适用于Python编译的二进制文件。

前提条件

- 已安装HPCKit，请参考**《Kunpeng HPCKit 26.0.RC1 安装指南》**。
- 已加载gcc版本环境变量，请参考**《Kunpeng HPCKit 26.0.RC1 安装指南》**中“安装卸载HPCKit > 设置环境变量 > 设置环境变量（setvars.sh方式）”。

6.2.1 普通代码段多副本配置步骤

步骤1 使用yum安装patchelf。

yum install patchelf

步骤2 建立numa-duplication.sh

```
#!/bin/bash

# Function to get all dependencies of an executable
function get_dependencies() {
    local path=$1
    for dep in $(ldd $path | awk '{print $3}' | grep -v 'not a dynamic executable'); do
        if [ -f $dep ]; then
            local resolved_dep=$(readlink -f $dep)
            if ! [[ " ${dependencies[*]} " =~ " ${dep} " ]]; then
                dependencies+=("$dep")
                get_dependencies $resolved_dep
            fi
        fi
    done
}

# Function to replace needed libraries in an executable
function replace_needed() {
    local elf_path=$1
    local numa_lib_path=$2
    local elf_dependencies=($(patchelf --print-needed $elf_path))
    for dep in "${elf_dependencies[@]}"; do
        if [ "$dep" == "ld-linux-aarch64.so.1" ]; then
            continue
        fi

        so_real_name=${dependencies_hash[$dep]}
        if [ -z "$so_real_name" ]; then
            echo "Error: Dependency $dep not found. $elf_path"
            exit 1
        fi

        patchelf --replace-needed "$dep" "$numa_lib_path/$so_real_name" $elf_path
    done
}

# Get the number of NUMA nodes
numa_node_num=$(numactl --hardware | grep "available:" | awk '{print $2}')

# Get the path of the application
app_path=$1
if [ ! -f "$app_path" ]; then
```

```
    app_path=$(which $app_path)
fi
app_name=$(basename $app_path)

# Declare and initialize global variables
declare -g dependencies=()
get_dependencies $app_path
echo ${dependencies[@]}

declare -A dependencies_hash
for dep in "${dependencies[@]}"; do
    so_name=$(basename $dep)
    dependencies_hash[$so_name]=$(basename $(readlink -f $dep))
done

# Copy the application to each NUMA node and replace dependencies
# rm -rf $app_name
pwd_path=$(pwd)
numa_node_num=16
pids=()
for i in $(seq 0 $((numa_node_num-1)))
do
    (
        path_numa="./noshared_libraries/numa_$i"
        path_numa_lib="./noshared_libraries/numa_$i/lib"
        mkdir -p $path_numa
        mkdir -p $path_numa_lib
        cp $app_path $path_numa
        replace_needed "$path_numa/$app_name" "$pwd_path/noshared_libraries/numa_$i/lib"
        for so_item in "${dependencies[@]}"
        do

            resolved_dep=$(readlink -f $so_item)
            cp $resolved_dep $path_numa_lib
            so_name=$(basename $resolved_dep)
            replace_needed "$path_numa_lib/$so_name" "$pwd_path/noshared_libraries/numa_$i/lib"
        done
    ) &
    pids+=($!)
done

for pid in "${pids[@]}"; do
    wait "$pid"
done
```

步骤3 运行numa-duplication.sh打包二进制, 生成一个noshared_libraries文件夹包含二进制以及依赖。

```
bash numa-duplication.sh /path_to_app/app
```

说明

- *path_to_app*表示最终要运行的文件*app*保存的路径，请根据实际情况替换。
- numa-duplication.sh和app应保存在同一文件夹下。

步骤4 建立运行脚本文件run.sh（以16进程16线程为例）。

```
#!/bin/bash


offset=2
cores=38        # cores per NUMA node
nppernuma=1
thdperproc=16

rank=${OMPI_COMM_WORLD_LOCAL_RANK}
numaid=$((rank / nppernuma))
numastart=$((numaid*cores))
mode=$((rank - numaid * nppernuma))
coreid=$((numastart + offset + mode * thdperproc))
```

 版权所有 © 华为技术有限公司

```
corend=$((coreid + thdperproc - 1))

#numa-duplication
fun_c=/path_to_app/app

if [[ -d noshared_libraries ]]; then
  fun_c=/path_to_app/noshared_libraries/numa_${numaid}/app
fi

numa_offset=16

# Provide the necessary runtime parameters at …
taskset -c ${coreid}-${corend} numactl -m $(($numa_offset+numaid)) ${fun_c} …
```

步骤5 多副本运行。

```
mpirun -x PATH -x LD_LIBRARY_PATH -bind-to none -np 16 -N 16 -x OMP_NUM_THREADS=16 run.sh
```

----结束

6.2.2 Python 代码段多副本配置步骤

步骤1 安装pyinstaller打包库。

```
yum install pyinstaller
```

步骤2 安装其他Python依赖库。

```
pip install pyyaml
pip install typing_extensions
pip install pympler
pip install ninja
yum install patchelf
```

步骤3 进入需要打包的Python应用根目录，建立文件夹lib_py，并将该应用所依赖的所有.so文件和数据文件放置在lib_py目录下。

步骤4 执行打包脚本py_install.sh。

```
#!/bin/bash
set -e

pyinstaller \
  --name=app_name \                          # Application name
  --onefile \
  --add-binary="lib_py/*.so:lib_py" \       # Binary files required
  --add-data=" lib_py:lib_py " \             # Data files required
  --hidden-import=app_name \                   # If the entry file of the Python application imports this application, this line is required; otherwise, please ignore it.

  --hidden-import=torch \
  app_entry.py                               # Entry python file

echo "onefile build complete! Binary: dist/app_name"
```

步骤5 建立rankfile。

```
#示例
rank 0=localhost slot=0-35
rank 1=localhost slot=38-73
rank 2=localhost slot=76-111
rank 3=localhost slot=114-149
rank 4=localhost slot=152-187
rank 5=localhost slot=190-225
rank 6=localhost slot=228-263
rank 7=localhost slot=266-301
```

步骤6 mpirun多副本运行（以8进程36线程应用为例），建立脚本run.sh并执行。

```
#!/bin/bash
export OMP_NUM_THREADS=36
```

```
export OMP_PROC_BIND=close
mpirun \
  -np 8 \
  -rf rankfile \
  -x OMP_NUM_THREADS \
  bash -c '
exec ./dist/app_name … '# Provide the necessary runtime parameters at …
```

补充：若运行时因内存消耗异常增长导致失败，请将步骤4中的--onefile修改为--onedir，并将运行脚本最后一行修改为：

```
exec ./dist/app_name/app_name … '# Provide the necessary runtime parameters at …
```

(该方式存在性能衰减的风险，请根据实际情况按需使用)。

----结束

 版权所有 © 华为技术有限公司

7 鲲鹏 920 专业版矩阵算力编程

在进行矩阵算力编程前，需要通过以下方式检测当前系统环境信息的正确性，确保支持矩阵算力。

- 查看硬件信息

  通过 lscpu 命令可以查看硬件的 flag 信息。

  ```
  lscpu | grep sme
  ```

  预期输出：

  ```
  Flags: fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp cpuid asimdrdm jscvt fcma lrcpc dcpop sha3 sm3 sm4 asimddp sha512 sve asimdfhm dit uscat ilrcpc flagm ssbs sb paca pacg dcpodp sve2 sveaes svepmull svebitperm svesha3 svesm4 flagm2 frint svei8mm svebf16 i8mm bf16 dgh ecv afp rpres sme smef64f64 smei8i32 smef16f32 smeb16f32 smef32f32 smefa64 wfxt
  ```

- 查看硬件矩阵算力向量长度

  查看 /proc/sys/abi/sme_default_vector_length 信息确认当前硬件的默认向量长度。

  ```
  sudo cat /proc/sys/abi/sme_default_vector_length
  64
  ```

  如果向量长度不为64则需要修改：

  ```
  sudo echo 64 > /proc/sys/abi/sme_default_vector_length
  ```

  说明

  如果执行代码后，如果出现段错误、程序性能突然变差、coredump、向量变量的后面几个元素结果不正确等现象，可能说明向量长度设置有误，此时需将系统向量长度设置为64。

7.1 矩阵编程指导

7.2 毕昇自动使能矩阵计算加速

7.3 对接HPCKit加速库使能矩阵计算加速

7.4 对接KUPL MMA使能矩阵计算加速

7.5 ARM Intrinsic使能矩阵计算加速

7.1 矩阵编程指导

矩阵计算是基于向量计算（Vector computation）定义的。在SVE的向量寄存器 (Z registers) 和谓词寄存器 (P registers) 之上，矩阵算力新增了一个 2D 累加器 (ZA accumulator)。

 版权所有 © 华为技术有限公司

矩阵算力是在可伸缩向量扩展（Scalable Vector Extensions和Vector computation）的基础上建立的，并增加了有效处理矩阵的能力，主要功能包括：

- 矩阵tile的存储
- 存取、插入、提取 tile向量，包括on-the-fly 转置
- 计算Vector computation向量的外积（Outer product）
- Streaming Vector computation模式

使能矩阵算力可通过以下四种方式实现：

- 毕昇编译器自动生成矩阵运算指令，完成矩阵运算的优化；
- 通过替换第三方库实现，例如将OpenBLAS替换为HPCKit组件KML中的KML_BLAS库或组件KuDNN库；
- 通过使用KUPL MMA 使能矩阵计算加速；
- 通过C/C++代码实现，例如核心算子使用矩阵算力 Intrinsic编写代码。

7.2 毕昇自动使能矩阵计算加速

为了提高矩阵计算相关指令在应用中的利用率，最新版本毕昇编译器加入了自动矩阵化能力，通过静态分析识别应用源码中存在的GEMM、SYR2K等典型计算模式，并获取行指标、列指标、维度等关键计算信息，在不影响程序正确性的前提下生成矩阵化的指令，提高程序性能。在应用程序编译时，可通过增加编译选项'-mcpu=hip11'、'-foptimize-loops'等来使能矩阵计算加速。举例如下：

编译命令：

```
clang -mcpu=hip11 -O1 -foptimize-loops gemm.c
```

源文件gemm.c的内容如下：

```
void gemm(int M, int N, int K, float ALPHA, float *A, int lda, float *B, int ldb, float *C, int ldc) {
    int i, j, k;
    for(i = 0; i < M; ++i){
        for(j = 0; j < N; ++j){
            float sum = 0;
            for(k = 0; k < K; ++k){
                sum += ALPHA * A[i * lda + k] * B[j + ldb * k];
            }
            C[i + ldc * j] += sum;
        }
    }
}
```

说明

1、编译优化等级要求>=O1；

2、可以通过增加编译选项'-S'生成编译后的汇编文件，检测是否存在'fmopa'指令以确认是否成功使能矩阵计算加速。

7.3 对接 HPCKit 加速库使能矩阵计算加速

7.3.1 对接鲲鹏数学库

HPCKit开发套件中的鲲鹏数学库KML提供了已使能矩阵计算的接口，如KML_BLAS库的Level3函数族。如果应用程序中使用了标准的一般矩阵乘矩阵接口gemm、标准的

三角矩阵方程求解接口trsm、标准的复数埃尔米特矩阵乘矩阵hemm、标准的实数对称矩阵乘矩阵接口symm等，可以直接对接KML_BLAS提供的接口以使能矩阵计算加速。更多鲲鹏数学库相关接口说明及使用指导可以参考鲲鹏社区的**《鲲鹏数学库 开发指南》**。

以使用OpenBLAS的双精度浮点数矩阵乘举证接口dgemm为例，仅需替换头文件<cblas.h>为<kblas.h>并调整编译指令链接KML_BLAS库即可：

对接前编译指令：

```
clang -O3 -o dgemm.o dgemm.c -lopenblas
```

对接后编译指令：

```
clang -O3 -o dgemm.o dgemm.c -lkblas
```

7.3.2 对接 KuDNN 算子库

HPCKit开发套件中主要用于AI框架的加速算子库KuDNN也提供了已使能矩阵算力的接口，如矩阵乘法计算的KuDNN::Gemm接口，该接口能够通过JIT动态代码技术根据输入规格生成相应的矩阵算力kernel，可以达到更好的性能。KuDNN::Gemm支持INT8、BF16、FP16和FP32四种类型。更多接口说明及使用指导可以参考鲲鹏社区的**《KuDNN开发指南》**

以使用OpenBLAS的单精度浮点数矩阵乘接口sgemm为例，说明如何进行替换

替换前源文件OpenBLAS_sgemm.c

```
#include <cblas.h>
void matrix_multiply(float *A, float *B, float *C, int m, int n, int k) {
    cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans, m, n, k, 1.0, A, k, B, n, 0.0, C, n);
}
```

替换后源文件KuDNN_sgemm.cpp

```
#include "kudnn.hpp"
void matrix_multiply(float *A, float *B, float *C, int m, int n, int k)
{
    // layout为AB
    using SizeType = KuDNN::SizeType;
    using Shape = KuDNN::Shape;
    using Type KuDNN::Element::TypeT;
    Shape srcShape(m, k);
    Shape weiShape(k, n);
    Shape dstShape(m, n);
    // Tensor初始化
    const KuDNN::TensorInfo srcTensor = {srcShape, Type::F32, KuDNN::Layout::AB};
    const KuDNN::TensorInfo weiTensor = {weiShape, Type::F32, KuDNN::Layout::AB};
    const KuDNN::TensorInfo dstTensor = {dstShape, Type::F32, KuDNN::Layout::AB};
    int numThreads = 1;
    // 构造算子
    KuDNN::Gemm gemmLayer(srcTensor, weiTensor, dstTensor, numThreads);
    // 执行算子
    gemmLayer.Run(A, B, C, 1.0, 0.0, numThreads);
}
```

替换后编译指令：

```
clang++ -O3 -o KuDNN_sgemm KuDNN_sgemm.cpp -lkudnn
```

除此之外，KuDNN算子库也可以通过插件的形式集成到PyTorch等AI框架中，具体方式可参考鲲鹏社区的**《KuDNN适配PyTorch接口说明》**

 版权所有 © 华为技术有限公司

7.4 对接 KUPL MMA 使能矩阵计算加速

KUPL矩阵编程模块提供了诸如D=A*B+C形式的MMA接口，便于发挥鲲鹏硬件的矩阵算力。其设计理念是KUPL矩阵编程上层MMA接口通过C++模板库实现多种矩阵计算Ops，底层则使能鲲鹏硬件的矩阵算力进行加速，从而达到屏蔽用户感知底层硬件能力的情况下快速使能加速。相较于鲲鹏数学库KML_BLAS和加速算子库KuDNN而言，KUPL矩阵编程MMA只提供Kernel Level的接口，具体在Gemm算子场景开发者既可以通过直接调用数学库BLAS Gemm接口加速也可以通过调用KUPL MMA接口+矩阵分块等优化手段加速；然后在矩阵乘融合算子场景开发者只能通过KUPL MMA接口加速。KUPL矩阵编程具体接口介绍参考：**《 KUPL矩阵编程接口函数概念说明 》**。

在此基于精度为Float64、Size为32*16*512的矩阵乘kernel算子为例，来介绍如何使用KUPL MMA接口来加速，具体代码示例如下：

```
#include "stdlib.h"

// 引用KUPL矩阵编程模块头文件并加载相应namespace
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    constexpr int MATRIX_K = 512;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_K);
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_K * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    /*
    创建KUPL Shape对象，用以描述计算矩阵的形状
    如A矩阵的size为32*512，则KUPL Shape对象一维大小应设置成32(表示行数)、二维大小应设置成512(表示列数)
    */
    auto shape_a = make_shape(Int<32>{}, Int<512>{});
    auto shape_b = make_shape(Int<512>{}, Int<16>{});
    auto shape_c = make_shape(Int<32>{}, Int<16>{});

    /*
    创建KUPL Stride对象，用以描述计算矩阵的各维度步长
    如A矩阵是32*512大小的列主序矩阵，则KUPL Stride对象一维大小应设置成1(表示同一列相邻两行的元素在原始buffer中下标差值为1)、二维大小应设置成32(表示同一行相邻两列的元素在原始buffer中下标差值为32)
    */
    auto stride_a = make_stride(Int<1>{}, Int<32>{});
    auto stride_b = make_stride(Int<16>{}, Int<1>{});
    auto stride_c = make_stride(Int<16>{}, Int<1>{});

    /* 创建KUPL Layout对象，包含Shape和Stride信息 */
    auto layout_a = make_layout(shape_a, stride_a);
    auto layout_b = make_layout(shape_b, stride_b);
    auto layout_c = make_layout(shape_c, stride_c);

    /* 定义KUPL mma和store方法从而供后续mma和store接口调用 */
    /*
    基于底层Ops按照atom_shape进行各维度计算尺寸扩展，从而确实mma行为
    如采用底层Ops为double精度下32x16x1的mma kernel，此处在M/N/K方向上atom_shape扩展分别为1/1/512，因此最终实现32x16x512的mma行为
    */
    auto mma_atom_shape = make_shape(Int<1>{}, Int<1>{}, Int<1>{});
    auto tiled_mma = make_tiled_mma(Ops<MMA_32x16x512_F64F64F64>{}, mma_atom_shape);
    /*
    基于底层Ops按照atom_shape进行各维度写回尺寸扩展，从而确定store行为
    如采用底层Ops为double精度下32x16的矩阵store kernel，此处在M/N方向上atom_shape扩展分别为1/1，因此最终实现32x16的矩阵store行为
    */
```

 版权所有 © 华为技术有限公司

```
    auto store_atom_shape = make_shape(Int<1>{}, Int<1>{});
    auto tile_store = make_tiled_store(Ops<STORE_32x16_F64>{}, store_atom_shape);

    /* 创建KUPL tensor对象纳管用户矩阵buffer */
    auto tensor_a = make_tensor(data_a, layout_a);
    auto tensor_b = make_tensor(data_b, layout_b);
    auto tensor_c = make_tensor(data_c, layout_c);

    /* 调用mma和store接口进行计算及写回，从而快速使能矩阵算力 */
    tensor_tiled_mma(tiled_mma, tensor_c, tensor_a, tensor_b, tensor_c);
    tensor_tiled_store(tile_store, tensor_c);

    free(data_a);
    free(data_b);
    free(data_c);
    return 0;
}
```

7.5 ARM Intrinsic 使能矩阵计算加速

Intrinsic编译是指将内置函数（intrinsic function）转换为机器代码的过程。内置函数是一些在编程语言中预定义的函数，它们通常用于执行一些底层操作或者优化特定的任务。在编译过程中，编译器会将这些内置函数转换为对应的机器指令，以便在运行时能够更高效地执行。同样地，ARM也提供了一组编译器内置的Intrinsic 函数，以便C/C++程序使能矩阵算力。

常用的矩阵算力使能的Intrinsic 函数可以分为4类，分别是2D 累加器ZA清空函数、READ函数、ADDHA/ADDVA函数以及MOPA/MOPS函数。具体的ARM Intrinsic函数可以在ARM官方查阅：**《 ARM Intrinsics 》**，也可以参考ARM 提供的 C/C++ 语言扩展标准文档：**《 Arm C Language Extensions (ACLE) 》**。

以下为一个简单的使用Arm intrinsic实现的一次单精度浮点数据向量外积操作示例：

```
void mopa(svfloat32_t v1, const float *src) __arm_streaming     // This function enters and returns in
streaming-SVE mode.
{
    svzero_za();                                        // Initialize ZA by zeroing all tiles.
    svbool_t p1 = svwhilelt_b32(0, 16);                         // Initialize predicate p1 with every element is active.
    svbool_t p2 = svwhilelt_b32(0, 16);                         // Initialize predicate p2 with every element is active.
    svfloat32_t v2 = svld1_f32(p2, src);                       // Load vector v2.

    svmopa_za32_m(/*tile*/0, p1, p2, v1, v2);                  // Sum of outer products and accumulate in ZA tile
0.

    for (int t = 0; t < 16; t++) {                          // write the ZA0 to output.
        svst1_hor_za32(0, t, p1, &output[t * 16]);              // svst1_hor_za32("0 for ZA0", "hor", p, "target
addr");
    }
    return;
}
```

更多不同数据类型的Arm intrinsic实现可以参考**hpckit-sample**开源仓中综合用例目录下的Arm intrinsic用例。

 版权所有 © 华为技术有限公司

8 鲲鹏 920 专业版 SDMA 编程

8.1 SDMA介绍及使能

8.2 基于KUPL实现

8.1 SDMA 介绍及使能

System Direct Memory Access（SDMA）是系统中负责数据的高带宽低延迟搬运的模块，是一种分布式分离式的DMA引擎。SDMA driver适配鲲鹏920专业版处理器的SDMA引擎，提供安全和性能两种模式，可满足多种应用场景需求。

SDMA为DMA的控制器，接受来自软件或者调度器的搬移任务，然后控制硬件完成搬移操作，最后通知软件的搬移完成状态，实现高带宽、低延迟的数据搬移。由于SDMA和其他device上的加速器是异步的，因此数据搬移可以在计算过程中完成，将数据搬移的开销掩盖在计算流水内，降低业务的非计算额外开销。

使能SDMA首先需要开启SMMU配置，具体开启方式参考上文提到的**SMMU配置**。

SDMA 组件分为用户态驱动和内核态驱动，使能系统中SDMA功能时，需要确保SDMA用户态驱动和内核态驱动加载正确。

通过下述命令查询SDMA内核态驱动是否加载：

```
ll /dev/sdma*
```

SDMA内核态驱动加载正确时，该路径下能够查询到4个SDMA设备，分别是sdma0、sdma1、sdma2、sdma3。

通过下述命令查询SDMA用户软件包是否正确安装：

```
ll /usr/lib64 | grep sdma
```

SDMA用户态软件包正确安装时，/usr/lib64目录下能够查询到libsdma_dk.so。

8.2 基于 KUPL 实现

KUPL的数据拷贝接口使能了SDMA数据搬运功能，通过封装SDMA驱动接口的方法，简化SDMA使用方法，降低学习成本。

具体地，能够通过KUPL相关数据拷贝接口，使用SDMA进行数据拷贝。举例如下：

 版权所有 © 华为技术有限公司

代码内容如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include "kupl.h"

int main()
{
        int height = 2, width = 200;
        int spitch = 300, dpitch = 400;
        char *src = (char*)kupl_malloc(KUPL_MEM_DEFAULT, sizeof(char) * height * spitch);
        char *dst = (char*)kupl_malloc(KUPL_MEM_DEFAULT, sizeof(char) * height * dpitch);
        for (int i = 0; i < height * spitch; i++) {
            src[i] = (char)i;
        }
        int ret = kupl_memcpy2d(dst, sizeof(char) * dpitch, src,
                sizeof(char) * spitch, sizeof(char) * width, height);
        assert(ret == KUPL_OK);
        kupl_free(KUPL_MEM_DEFAULT, src);
        kupl_free(KUPL_MEM_DEFAULT, dst);
        return 0;
}
```

说明

1、上述示例演示了使用KUPL数据拷贝接口进行二维内存拷贝的流程。

2、上述kupl_memcpy2d函数将src数组中的内容复制到dst数组，其中src位置的数据每取出width数据的偏移量为300，dst位置的数据每存储width数据的偏移量为400，拷贝数据的宽和高分别为200和2。

3、kupl_memcpy2d接口能够通过SDMA进行二维数据的同步拷贝；同理，可以通过kupl_memcpy、kupl_memcpy_async、kupl_memcpy2d_async等接口进行一维同步、一维异步、二维异步数据拷贝，接口的详细信息和使用方法可以参考鲲鹏社区的**《 KUPL开发指南 》**。

 版权所有 © 华为技术有限公司

9 鲲鹏 920 专业版负载测试

9.1 Benchmark测试

9.2 应用分析

9.1 Benchmark 测试

9.1.1 DGEMM 测试

9.1.1.1 工具介绍

BLAS-Tester是一款用于测试BLAS库中各函数性能的工具，它将结果与Netlib引用BLAS进行比较。

前提条件

- 已安装HPCKit，请参考**《Kunpeng HPCKit 26.0.RC1 安装指南》**。
- 已加载GCC版本环境变量，请参考**《Kunpeng HPCKit 26.0.RC1 安装指南》**中“安装卸载HPCKit > 设置环境变量 > 设置环境变量（setvars.sh方式）”。

9.1.1.2 工具安装

操作步骤

步骤1 使用PuTTY工具，以root用户登录服务器。

步骤2 上传IMB安装包“**BLAS-Tester-master.zip**”到服务器目录下，如“/share/software/tools”。

步骤3 执行以下命令解压IMB安装包。

cd /share/software/tools

unzip BLAS-Tester-master.zip

步骤4 加载gcc版本环境变量，请参考**《Kunpeng HPCKit 26.0.RC1 安装指南》**中“安装卸载HPCKit > 设置环境变量 > 设置环境变量（setvars.sh方式）”。

 版权所有 © 华为技术有限公司

步骤5 进入BLAS-Tester-master目录。

cd BLAS-Tester-master

步骤6 执行以下命令编译BLAS-Tester

make TEST_BLAS=${*INSTALL_PATH*}/HPCKit/latest/kml/gcc/lib/sme/kblas/multi/libkblas.so ARCH=aarch64 GENbin=${*BIN_PATH*} NUMTHREADS=8 L2SIZE=524288 -j ONLY_PERFORMANCE=${*PERF*}

INSTALL_PATH：HPCKit的安装目录。

BIN_PATH：输出二进制文件目录地址。

PERF：是否只测性能，如果为0则附带功能测试。

步骤7 cd ${BIN_PATH}目录下，查看输出文件为以下二进制文件。

```
xcl1blastst  xcl2blastst  xcl3blastst  xdl1blastst  xdl2blastst  xdl3blastst  xsl1blastst  xsl2blastst  xsl3blastst  xzl1blastst  xzl2blastst  xzl3blastst
```

----结束

9.1.1.3 测试运行

步骤1 进入到生成的二进制目录。

cd ${BIN_PATH}

步骤2 加载gcc版本环境变量，请参考《Kunpeng HPCKit 26.0.RC1 安装指南》中“安装卸载HPCKit > 设置环境变量 > 设置环境变量（setvars.sh方式）”。

设置BLAS的并行执行线程数为4。

export OMP_NUM_THREADS=4

设置运行时线程按照整体亲和性依次进行绑核，一个线程固定在一个核上。

export OMP_PROC_BIND=close

步骤3 使用的时候，执行对应的数据类型【s/d/c/z】和对应的level【1/2/3】的二进制文件。在后面跟对应的参数即可得到结果，例如：

taskset -c 1-36 ./xdl3blastst -R gemm -m 4096 -n 9216 -k 4096 -T 0 -F 1000000

此处调用数据类型为double【d】的level3接口，接口名为dgemm，计算的规模为m=4096,n=9216,k=4096，只测试接口性能。

步骤4 查看回显，得到的结果如图，图中需要关注的结果为红框的结果，代表库中dgemm的算力。测试结果在 400000MFLOP 左右说明执行成功。

```
[root@cn00002 bin_multi]# taskset -c 1-36 ./xdl3blastst  -R gemm -m 4096 -n 9216 -k 4096 -T 0 -F 1000000


./xdl3blastst -R gemm -m 4096 -n 9216 -k 4096 -T 0 -F 1000000


-------------------------------- GEMM ---------------------------------
TST# A B    M    N    K ALPHA  LDA  LDB  BETA  LDC  TIME MFLOP SpUp  TEST
==== = = ==== ==== ==== ===== ==== ==== ===== ==== ===== ===== ==== =====
   0 N N 4096 9216 4096   1.0 4096 4096   1.0 4096  0.00 432345564227.6 1.00 -----
   0 N N 4096 9216 4096   1.0 4096 4096   1.0 4096  0.76 405931.1 0.00 SKIP

1 tests run, all skipped
```

 版权所有 © 华为技术有限公司

步骤5 使用`./xdl3blastst -h`可以查看各种参数。

```
[root@cn00002 bin_multi]# ./xdl3blastst -h


./xdl3blastst -h

ATLAS usage:
    ./xdl3blastst [-options ...]

where options include:
   -h                                  . print this message
   -R <rout>                           . select  one  or all routines to test.
                                         rout must be in {all, gemm,symm,syrk,
                                         syr2k,trmm,trsm}.
                                         Default: -R gemm.  Ex: -R trmm
   -R <nrout> <rout1> ... <routN>      . same as above for more than one rout-
                                         tine. Ex: -R 3 genn trmm symm
   -S <nside>  L/R                     . select values for the SIDE parameter.
                                         Default: -S 1 L. Ex: -S 2 L R
   -U <nuplo>  L/U                     . select values for the UPLO parameter.
                                         Default: -U 1 L. Ex: -U 2 L U
   -A <ntrans> N/T/C                   . select values of the TRANSA parameter.
                                         Default: -A 1 n. Ex: -A 2 N T
   -B <ntrans> N/T/C                   . select values of the TRANSB parameter.
                                         Default: -B 1 N. Ex: -B 2 N T
   -D <ndiags> N/U                     . select values for the DIAG parameter.
                                         Default: -D 1 N. Ex: -Diag 2 N U
   -m <m>                              . select one value for the parameter M.
                                         Ex: -m 100
   -n <n>                              . same as above for the parameter N.
   -k <k>                              . same as above for the parameter K.
   -M <m1>  <mN>  <minc>               . select the values of M, from m1 to mN
                                         by increment of minc. m1 > 0.
                                         Ex: -M 100 1000 100
   -N <n1>  <nN>  <ninc>               . same as above for the values of N.
   -K <k1>  <kN>  <kinc>               . same as above for the values of K.
   -a <nalphas> <a1> ... <aN>          . select the values of ALPHA.  Default:
                                         -a 1 1.0. Ex: -a 3 -1.0 0.0 1.0
   -b <nbetas>  <beta1>  ... <betaN>   . same as above for the parameter BETA.
   -d                                  . use smallest possible leading  dimen-
                                         sion for the array A.
                                         Default: max( mN, nN, kN ).
   -T <0/1>                            . disable/enable computational check.
                                         Default: -T 1
   -F <mflops>                         . perform at least mflops per measure.
   -C <CacheSize>                       . select how much memory in kilobytes
                                         to flush to clear the caches. Put 1
                                         for no cache flushing. Default is the
                                         size of the biggest cache.
```

----结束

9.1.2 STREAM 测试

9.1.2.1 工具介绍

STREAM是业界广为流行的综合性内存带宽实际性能测量工具之一。随着处理器处理核心数量的增多，内存带宽对于提升整个系统性能越发重要，如果某个系统不能足够迅速地将内存中的数据传输到处理器当中，若干处理核心就会处于等待数据的闲置状态，而这其中所产生的闲置时间不仅会降低系统的效率还会抵消多核心和高主频所带来的性能提升优势。STREAM具有良好的空间局部性，是对TLB友好、Cache友好的一款测试。STREAM支持Copy、Scale、Add、Triad四种操作。

9.1.2.2 工具安装

步骤1 下载**STREAM 软件包**。

步骤2 使用gcc编译stream.c，编译宏STREAM_ARRAY_SIZE为测试数组大小，NTIMES为测试次数，参考编译命令：

```
gcc -O3 -fopenmp -DSTREAM_ARRAY_SIZE=141648512 -DNTIMES=200 stream.c -o stream_c.exe
```

 版权所有 © 华为技术有限公司

编译后得到的 stream_c.exe 即为可执行测试文件。

----结束

9.1.2.3 片上内存带宽测试

本文针对片上内存带宽提供两种测试方式，即单NUMA 片上内存带宽测试和单节点片上内存带宽测试。

以下是单NUMA 片上内存带宽测试方式：

步骤1 将测试线程数设置为单NUMA支持的最大线程数，并通过绑定片上内存以便将数据分布到片上内存。

步骤2 测试单NUMA的片上内存带宽，参考测试命令如下：

```
OMP_NUM_THREADS=38 OMP_PROC_BIND=close taskset -c 0-37 numactl -m 16 ./stream_c.exe
```

参考测试结果中的Triad值，单个NUMA的带宽约在380000~410000MB/s：

```
-------------------------------------------------------------
STREAM version $Revision: 5.10 $
-------------------------------------------------------------
This system uses 8 bytes per array element.
-------------------------------------------------------------
Array size = 141648512 (elements), Offset = 0 (elements)
Memory per array = 1080.7 MiB (= 1.1 GiB).
Total memory required = 3242.1 MiB (= 3.2 GiB).
Each kernel will be executed 500 times.
 The *best* time for each kernel (excluding the first iteration)
 will be used to compute the reported bandwidth.
-------------------------------------------------------------
Number of Threads requested = 38
Number of Threads counted = 38
-------------------------------------------------------------
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 6192 microseconds.
   (= 6192 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-------------------------------------------------------------
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           402552.3     0.005893     0.005630     0.007208
Scale:          380128.4     0.006136     0.005962     0.009997
Add:            420949.0     0.008377     0.008076     0.009797
Triad:          403806.4     0.008782     0.008419     0.010945
-------------------------------------------------------------
Solution Validates: avg error less than 1.000000e-13 on all three arrays
-------------------------------------------------------------
```

----结束

以下是单节点片上内存带宽测试方式：

步骤1 使用MPI启动16个进程，每个进程绑定一个NUMA并将进程的线程数设置为单NUMA支持的最大线程数，同时通过绑定对应的片上内存以便将进程的数据分布到片上内存。

步骤2 测试单节点的片上内存带宽，参考测试命令如下：

```
mpirun -np 16 --map-by numa ./bw_node_mem_on_chip.sh
```

bw_node_mem_on_chip.sh脚本参考如下：

 版权所有 © 华为技术有限公司

```
#!/bin/bash
rank=${OMPI_COMM_WORLD_LOCAL_RANK}
start=$(($rank * 38))
end=$(($rank * 38 + 37))
OMP_NUM_THREADS=38 OMP_PROC_BIND=close taskset -c ${start}-${end} numactl -m $(($rank + 16)) ./
stream_c.exe
```

参考测试结果中的Triad值，每个进程的测试结果约在380000~410000MB/s，则单节点片上内存带宽为16个进程带宽测试结果的总和：

```
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           395880.0     0.006105     0.005725     0.012895
Scale:          383998.0     0.006379     0.005902     0.013061
Add:            414187.1     0.008566     0.008208     0.016977
Triad:          399149.2     0.008950     0.008517     0.014249
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           401481.2     0.006033     0.005645     0.011622
Scale:          382253.1     0.006311     0.005929     0.018508
Add:            421359.5     0.008711     0.008068     0.021030
Triad:          408012.3     0.008970     0.008332     0.019766
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           394696.5     0.006160     0.005742     0.022151
Scale:          386417.5     0.006283     0.005865     0.010724
Add:            414235.3     0.008654     0.008207     0.023914
Triad:          398357.4     0.009105     0.008534     0.029001
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           396987.7     0.006275     0.005709     0.054602
Scale:          380966.3     0.006411     0.005949     0.012462
Add:            413071.2     0.008757     0.008230     0.058634
Triad:          397979.4     0.009042     0.008542     0.016576
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           395123.1     0.006204     0.005736     0.013740
Scale:          380645.9     0.006716     0.005954     0.056543
Add:            414127.0     0.008792     0.008209     0.042559
Triad:          398680.4     0.009151     0.008527     0.018464
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           397751.8     0.006195     0.005698     0.014172
Scale:          383224.0     0.006464     0.005914     0.025427
Add:            414825.8     0.008741     0.008195     0.023310
Triad:          400382.1     0.009250     0.008491     0.031880
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           395599.9     0.006272     0.005729     0.055929
Scale:          387283.4     0.006598     0.005852     0.019510
Add:            414427.9     0.008822     0.008203     0.057628
Triad:          398168.3     0.009365     0.008538     0.024487
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           395106.6     0.006227     0.005736     0.012475
Scale:          383424.9     0.006604     0.005911     0.135234
Add:            415236.5     0.008757     0.008187     0.015858
Triad:          400236.0     0.009139     0.008494     0.016357
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           394565.4     0.006253     0.005744     0.021472
Scale:          381486.1     0.006486     0.005941     0.012711
Add:            413922.6     0.008824     0.008213     0.026116
Triad:          398446.5     0.009428     0.008532     0.054950
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           396011.9     0.006345     0.005723     0.055082
Scale:          382053.4     0.006477     0.005932     0.037756
Add:            415951.2     0.008905     0.008173     0.059440
Triad:          399574.2     0.009203     0.008508     0.018427
```

 版权所有 © 华为技术有限公司

```
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           395665.8     0.006477     0.005728     0.029760
Scale:          383471.3     0.006535     0.005910     0.016319
Add:            413778.5     0.008959     0.008216     0.023425
Triad:          399899.2     0.009258     0.008501     0.018282
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           396491.0     0.006429     0.005716     0.020900
Scale:          383224.0     0.006531     0.005914     0.029009
Add:            415284.9     0.008942     0.008186     0.023462
Triad:          397846.2     0.009269     0.008545     0.023168
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           403714.9     0.006431     0.005614     0.053611
Scale:          378237.7     0.006667     0.005992     0.055854
Add:            421995.5     0.009127     0.008056     0.058290
Triad:          403315.2     0.009863     0.008429     0.134397
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           404058.1     0.006639     0.005609     0.024329
Scale:          384463.9     0.006897     0.005895     0.056308
Add:            420477.3     0.009536     0.008085     0.042015
Triad:          406546.5     0.009711     0.008362     0.043979
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           397186.8     0.006756     0.005706     0.055778
Scale:          376350.9     0.006862     0.006022     0.024000
Add:            413682.4     0.009218     0.008218     0.056509
Triad:          399160.4     0.009873     0.008517     0.035960
-------------------------------------------------------------
Function    Best Rate MB/s  Avg time     Min time     Max time
Copy:           394434.5     0.006698     0.005746     0.057386
Scale:          384775.2     0.007042     0.005890     0.055602
Add:            414319.5     0.009339     0.008205     0.059440
Triad:          398491.0     0.009642     0.008531     0.024842
-------------------------------------------------------------
```

----结束

9.2 应用分析

本章介绍如何分析应用使用片上内存和矩阵算力编程后的优化结果。

9.2.1 片上内存使用分析与性能优化

本节介绍如何确定片上内存的使用方式及片上内存带宽利用率以提高程序性能。

步骤1 运行目标程序，执行如下命令观察内存使用情况。

1. 执行如下命令，查看内存的增长及SWAP使用情况，此时查看的MEM包含片上内存和DDR，无法明确不同NUMA对应的是片上内存还是DDR。

   free -h

   回显如下。

   ```
                  total        used        free      shared  buff/cache   available
   Mem:           567Gi        93Gi       425Gi       156i        67Gi       473Gi
   Swap:            0B          0B          0B
   ```

2. 执行如下命令，可以看到具体的NUMA node的内存使用情况。

   numactl -H

   如下图所示1-15 NUMA为DDR，每个NUMA 32G；16 -31 NUMA 为片上内存，每个NUMA 4G

```
...
node 14 cpus: 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548
549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568
569
node 14 size: 32250 MB
node 14 free: 28237 MB
node 15 cpus: 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586
587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606
607
node 15 size: 32231 MB
node 15 free: 28098 MB
node 16 cpus:
node 16 size: 4096 MB
node 16 free: 4085 MB
node 17 cpus:
node 17 size: 4096 MB
node 17 free: 4089 MB
node 18 cpus:
node 18 size: 4096 MB
node 18 free: 4090 MB
```

步骤2 确定片上内存的使用方式。

- 如果整体使用内存量小于目标大小（平均单NUMA使用小于4GB），则可以直接使用**numactl -m**指定片上内存。
- 如果整体使用内存大于目标大小（平均单NUMA使用大于4GB），则需要选择采用如下方式之一申请内存：1. 使用cache模式，具体开启方式参考上文提到的**5.2-BIOS配置-片上内存模式设置**; 2. 使用**numactl -P**指定片上内存，超出部分会分配在DDR内存; 3. 修改代码使用上文提到的**方法二：使用KUPL接口申请片上内存**或**方法三：使用memkind接口申请片上内存**。

步骤3 抓取片上内存带宽利用情况。

应用运行过程中可以通过鲲鹏DevKit的系统性能分析工具Tuner抓取片上内存带宽利用情况。参考命令如下：

```
devkit tuner memory -m 4
```

上述指令执行结束后，可输出内存带宽情况，如下内容所示：

```
Memory Summary Report-ALL                              Time:2026/04/14 14:30:54
================================================================================
System Information
--------------------------------------------------------------------------------

Linux Kernel Version  5.10.0-285.2.v2403.ky10p.aarch64
Cpu Type              Kunpeng 920 72Fx
NUMA NODE(cpus)       0(0-37)     1(38-75)    2(76-113)   3(114-151)
                      4(152-189)  5(190-227)  6(228-265)  7(266-303)
                      8(304-341)  9(342-379)  10(380-417) 11(418-455)
                      12(456-493) 13(494-531) 14(532-569) 15(570-607)
                      16(N/A)     17(N/A)     18(N/A)     19(N/A)
                      20(N/A)     21(N/A)     22(N/A)     23(N/A)
                      24(N/A)     25(N/A)     26(N/A)     27(N/A)
                      28(N/A)     29(N/A)     30(N/A)     31(N/A)
Percentage of core Cache miss
--------------------------------------------------------------------------------

L1D         3.40%
L1I         0.08%
L2D         74.90%
L2I         65.45%

DDR Bandwidth (system wide)
--------------------------------------------------------------------------------

ddrc_write        2710.97MB/s
ddrc_read         8881.06MB/s
```

 版权所有 © 华为技术有限公司

```
HBM Bandwidth
-------------------------------------------------------------------------------
HBM read        4512924.00MB/s
HBM write       5458727.00MB/s
```

说明

1、鲲鹏DevKit的系统性能分析工具Tuner的参数说明及更多使用方式可以参考官网指导：**《系统性能分析Tuner》**

2、鲲鹏DevKit的系统性能分析工具Tuner抓取的是整个节点的片上内存总带宽。

----结束

9.2.2 矩阵算力使用分析与性能优化

本节介绍如何通过分析业务中矩阵计算使用情况，提高程序对矩阵算力指令利用率，提升整体性能。

步骤1 使用DevKit tuner工具的HPC应用分析特性抓取目标程序的矩阵算力指令使用情况，执行如下命令：

```
mpirun -np 1 devkit tuner hpc-perf -o /tmp/hpc-perf-detail.tar ./a.out
```

其中a.out为应用程序的二进制文件。采集过程中会输出如下内容：

```
================================================================================
Version     : DevKit 26.0.RC1
CPU Model   : Kunpeng 920 72F8
Command     : devkit tuner hpc-perf -o /tmp/hpc-perf-detail.tar ./a.out
================================================================================
=====================================PARAM
CHECK=====================================
PARAM CHECK success.
==================================PREPARE FOR
COLLECT===================================
preparation of collection success.
[WARNING]Operation Tips:
1. During the collection, the /tmp directory is used to store temporary files. Ensure that this directory has
sufficient space.
2. If the collection is suspended, data may be lost. Ensure that the collection process is complete.
================================COLLECT AND ANALYSIS
================================
...
...
The report /tmp/hpc-perf-detail.tar is generated successfully
To view the summary report，you can run: devkit report -i /tmp/hpc-perf-detail.tar
To view the detail  report，you can import the report to WebUI or IDE
=====================================RESTORE
CLUSTER====================================
restore cluster success.
========================================FINISH
========================================
```

步骤2 程序运行结束后，会将分析结果保存至/tmp/hpc-perf-detail.tar，可通过devkit report工具查看分析结果，执行命令如下：

```
devkit report -i /tmp/hpc-perf-detail.tar
```

返回信息如下：

```
================================================================================
Version     : DevKit 26.0.RC1
CPU Model   : Kunpeng 920 72F8
Command     : devkit tuner report -i /tmp/hpc-perf-detail.tar
================================================================================
Elapsed Time                    :           2.2428 s (rank 000, jitter = 0.0000s)
```

 版权所有 © 华为技术有限公司

```
CPU Utilization                 :              0.08 % (0.48 out of 608 CPUs)
  Cycles per Instruction (CPI)  :               0.7606
  Instructions Retired          :           2824211908
MPI
===
  Top Hotspots on MPI Critical Path
  No data. Re-collect with -L detail --critical-path
  Top MPI Critical Path Segments
  No data. Re-collect with --critical-path
Instruction Mix
===============
  Memory                        :              58.57 %
    Load                        :              56.30 %
    Store                       :               7.30 %
  Scalar                        :              23.85 %
    Integer                     :              23.19 %
    Floating Point              :               0.66 %
  Vector                        :               3.76 %
    Advanced SIMD               :               0.00 %
    SVE (+ loads/stores)        :               3.46 %
    SME (retired)               :               0.30 %
      Integer                   :               0.00 %
      Floating Point            :               0.30 %
  Crypto                        :               0.00 %
  Branches                      :              13.29 %
    Immediate                   :              13.28 %
    Indirect                    :               0.01 %
    Return                      :               0.01 %
  Barriers                      :               0.00 %
    Instruction Synchronization :               0.00 %
    Data Synchronization        :               0.00 %
    Data Memory                 :               0.00 %
  Not Retired                   :               3.43 %

Top-down
========
  Retiring                      :              32.87 %
  Backend Bound                 :              59.30 %
    Memory Bound                :              23.92 %
      L1 Bound                  :               4.14 %
      L2 Bound                  :               0.92 %
      L3 or DRAM Bound          :              18.86 %
      Store Bound               :               0.00 %
    Core Bound                  :              35.39 %
  Frontend Bound                :               6.66 %
  Bad Speculation               :               1.17 %

Memory subsystem
================
  Average DRAM Bandwidth        :          3.0807 GB/s
    Read                        :          2.4468 GB/s
    Write                       :          0.6339 GB/s
  Max Bandwidth                 :         15.0965 GB/s
  Bandwidth bound               :              0.00 %
    Top Hotspots within High Bandwidth
    No data. Re-collect with -L detail
  No data to show
```

通过分析报告中的SME (retired)数值判断应用对矩阵算力的使用情况。更多DevKit的应用性能分析功能可参考DevKit官网：**《 系统性能分析Tuner-HPC应用分析 》**

----结束

 版权所有 © 华为技术有限公司